Computer Networks 225 (2023) 109659

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

Auto-tune: An efficient autonomous multi-path payment routing algorithm for Payment Channel Networks

Hsiang-Jen Hong *, Sang-Yoon Chang *, Xiaobo Zhou

Department of Computer Science, University of Colorado at Colorado Springs, Colorado Springs, 80918, CO, USA

ARTICLE INFO

Keywords:
Blockchain
Cryptocurrency
Multi-path payment
Payment Channel Network
Routing algorithm
Bitcoin Lightning Network

ABSTRACT

Payment Channel Network (PCN) is a scaling solution for Cryptocurrency networks. We advance the PCN multi-path routing by better modeling the system and incorporating the cost of routing fee and the privacy requirement of the channel balance. We design an autonomous routing algorithm, Auto-Tune, optimizing the routing concerning the success rate and the routing fee and utilizing the limited channel capacity information. The simulation result shows a significant performance gain in the success rate of Auto-Tune over the current PCN implementation based on single-path routing. To analyze the performance cost from the system requirement of the channel balance privacy, we compare Auto-Tune against the state-of-the-art Flash algorithm assuming the availability of the channel-balance information (such channel-balance information violates the PCN privacy requirement and does not comply with the current PCN implementations and practices). The simulation results show that the success rate and fee obtained by Auto-Tune are close to that obtained by Flash (which achieves the optimal fee result by using the exact channel-balance information).

1. Introduction

Cryptocurrency is a next-generation networking application revolutionizing financial processing (which traditionally relies on centralized authorities and is therefore subject to censorship). The cryptocurrency market is worth more than $3 trillion now, and Bitcoin is the fastest asset to reach $1 trillion (1.2 trillion dollars as of today[1]). However, the consensus algorithms used to confirm each transaction limits the throughput of cryptocurrencies. Cryptocurrency processes the transactions at a low transaction rate, depending on different implementations (e.g., Bitcoin [1] processes seven transactions per second. Ethereum [2] processes fifteen transactions per second), which is in contrast to the fiat currencies processing tens of thousands of transactions per second such as Visa network. Payment Channel Network (PCN) is an established and effective scaling solution for the cryptocurrency network by using bidirectional payment channels to enable faster and cheaper transactions. PCN is a second layer network operating on top of the cryptocurrencies network enabling off-chain transactions. The off-chain transactions can be efficiently performed with a local database between two parties without broadcasting to the public blockchain. In contrast, on-chain transactions need to be broadcast to the network and validated before adding it to the distributed ledger, which can take approximately an hour. In PCN, most of the transactions can be conducted off-chain. Only the funding transaction (the first transaction determining the channel balance) and the settlement transaction (the last transaction for closing the channel and obtaining their shares of the multi-sig wallet) should be broadcast and mined on the blockchain. Several PCN systems have been designed, such as Lightning Network for Bitcoin [3], and Raiden Network for Ethereum [4].

All payment channels comprise a PCN. By using PCN, a sender can execute a transaction with any receiver in the network with the help of the intermediate nodes in forwarding the transaction. In other words, the sender does not need to directly establish the payment channel with the receiver, which requires a much higher cost than the routing fee paid to those intermediate nodes. As mentioned in [5], the estimated cost for opening and closing one Lightning Network channel is 1.72×10^4 satoshis. In contrast, the routing fee for forwarding a transaction by single path routing with the maximum amount (43×10^6 satoshis) while considering the average hops in reaching other nodes measured in [6] is only 330 satoshis with the default fee model which is around 52 times smaller than not using PCN and directly opening the channel to the receiver.

Unfortunately, payment reliability (success rate) is a concern for current PCN implementations [7–9]. Two key factors are affecting the reliability of sending payments. First, a sender node has the view of network topology. However, it only has channel capacity information (i.e., the sum of the channel balances) for helping with the routing

* Corresponding authors.
E-mail addresses: hhong@uccs.edu (H.-J. Hong), schang2@uccs.edu (S.-Y. Chang).

[1] https://coinmarketcap.com/

https://doi.org/10.1016/j.comnet.2023.109659
Received 21 July 2022; Received in revised form 4 January 2023; Accepted 21 February 2023
Available online 24 February 2023

decision. Current deployed PCNs have not revealed channel balances for privacy concerns, e.g., an attacker can utilize such information to conduct attacks that can harm the PCN, such as the LockDown attack [10]. Second, the current routing algorithm is based on single-path routing. With single-path routing, a transaction with large payment amounts could be hard to find a path to support this payment. To help improve reliability, the R&D community of PCN works on techniques for supporting multi-path payments [11,12]. Such an approach can be expected to improve the payment success rate. However, more paths selected for sending a single payment yield more routing fees. Given this, the routing algorithm design should consider the trade-off between the success rate and routing fee while utilizing the limited channel capacity information.

In this work, we present an efficient autonomous routing algorithm called Auto-Tune to address the path selection and payment splitting for PCN multi-path routing. Our Auto-Tune algorithm is autonomous because it makes its own decision about whether a payment should be split based on the dynamic and implicit network view with limited channel capacity information. In addition, our algorithm tunes/controls the best split amounts allocated among paths for payment to optimize the success rate and the routing fee. In more detail, Auto-Tune selects the candidate paths and sorts those paths according to a metric called "path score" based on path success probability. With the paths sorted by scores, the algorithm autonomously selects a feasible path set with the highest scores and finds the best payment splitting among these paths to maximize the success probability. Moreover, our algorithm takes a split payment's success/failure results as feedback to help make a better routing decision and improve the success rate. To optimize the success rate, we propose using the out-of-band (OOB) channel (existing in the current PCN to share the payment invoice) to share the last-hop balance information. With such information, more accurate path scores can be computed. It thus helps with path selection and improves performance.

The rest of the paper is organized as follows. Section 2 discusses the background of PCN. Section 3 presents the motivation of this work and includes a motivation example to show the importance of multi-path payment routing. Section 4 introduces the model and assumptions of this work. Section 7 reviews the related work. Section 5 presents the design of our proposed Auto-Tune algorithm in detail. Section 6 conducts the performance comparison based on simulation among Auto-Tune algorithms (including the cases that Auto-Tune with OOB and Auto-Tune without OOB), the shortest path approach used in current PCN implementations, and the state-of-art Flash [13] algorithm. Section 8 draws a conclusion.

2. Background

2.1. Scalability of cryptocurrency

Cryptocurrencies have attracted growing interest from both investors and researchers. However, the low throughput of cryptocurrency networks has hindered the scalability of cryptocurrency systems [14,15]. As we mentioned in Section 1, Bitcoin processes only seven transactions per second, and Ethereum processes 15 transactions per second. Such an unfortunate situation arises because of some restrictions in the blockchain. Each transaction needs to be validated and written into a block. In addition, it takes time (10 min for Bitcoin) for a block to be created. Therefore, many transactions will be queued and wait to be included in a block. Several solutions have been proposed to deal with the scalability issue [16,17]. Payment Channel Network (PCN) is one of the promising scaling solutions established and used in real-world implementations. It utilizes bidirectional payment channels to enable faster and cheaper transactions. PCN such as Lightning Network has seen rapid growth over recent years. As measured by txstats.com,[2] the Lightning Network holds around 3.4 K BTC as of today (which is around $130M USD) and still keeps increasing.

[2] https://txstats.com/dashboard/db/lightning-network

2.2. Payment channel establishment

Two parties establish a payment channel by depositing funds on a 2-of −2 multi-signature wallet. Assume that i and j deposit the funds to the channel C_{ij}. A funding transaction is created accordingly as the first transaction determining the balances of the channel. We denote the balance of i and j on C_{ij} are b_{ij} and b_{ji} respectively. The balances on C_{ij} keep changing as long as the parties exchange mutually signed commitment transactions to modify the balance of the channel. Specifically, i can make a payment to j with a payment amount a_{ij} only if the balance is sufficient to support the payment, i.e., $b_{ij} \geq a_{ij}$. Such payment is accomplished by a commitment transaction modifying the balances accordingly. That is, $b_{ij} = b_{ij} - a_{ij}$ and $b_{ji} = b_{ji} + a_{ij}$. Suppose the two parties decide to close the channel. In that case, both parties use the most recent balance sheet to pay their share in the 2-of −2 multi-signature wallet by broadcasting a settlement transaction. By using the payment channel, as opposed to sending all transactions to the blockchain and waiting to be included into the blockchain (as discussed in Section 2.1), only the initial funding transaction and the finalized settlement transaction should be on-chain. The remaining commitment transactions are processed off-chain. That is why PCN can significantly improve the transaction throughput.

2.3. Public channel capacity vs. Private channel balance

In current deployed PCNs, the channel balances mentioned in the previous section are only known to the two parties establishing the channel. Such information is not revealed to the public due to privacy concerns [5,18–20]. Specifically, disclosing such data to the public could increase vulnerability, such as the Lockdown attack [10]. In Lockdown attack, the attacker utilizes the exact balance information to block a victim as a middle node in multi-path payments and obtains a dominant position in Lightning Network. The current designs thus only share the channel capacity, i.e., the sum of channel balances ($c_{ij} = b_{ij} + b_{ji}$) for each channel C_{ij}, to nodes in PCN.

2.4. PCN characteristics

The layer-2 PCN differs from the layer-1 Cryptocurrency network or the traditional computer network in the following two aspects: (1) PCN is relatively stable. Opening or closing a payment channel requires on-chain transactions. A channel (link) thus usually remains in the network after the establishment. (2) The channel capacity in PCN is defined as the sum of channel balances as opposed to the capacity determined as bandwidth in the traditional computer network. The channel capacity is fixed in PCN because the users cannot directly top-up balances and change the channel capacity in the current PCN design. Specifically, the channel capacity is fixed after creating the funding transaction specifying the balances. Only the channel balances will be adjusted by commitment transactions as discussed in Section 2.2.

2.5. Payment routing and fee model

For requesting a payment sent from i to j, j will create an invoice to request a payment sent from i. The invoice includes the payment hash, the payment amount, etc. After i receives the invoice from the j, i can decide to build a payment channel with j or select a routing path for forwarding this payment. As discussed in Section 1, the cost of using a routing path to forward a payment is significantly lower than the cost of building a direct payment channel because of the current routing fee model. Therefore, a sender prefers sending a payment by a routing path rather than establishing a new payment channel. For reader's reference, the fee model in the Lightning Network is computed $f_b + a_{ij} \times f_a$, where f_b and f_a are the base fee and additional fee and a_{ij} is the payment amount. The default values of f_b an f_a are 1 satoshi and 10^{-6} satoshi respectively. As discussed in Section 2.3, for making

a routing decision, i only has the information of network topology and the channel capacities. Therefore, a sender cannot determine whether a selected routing path has sufficient funds to route a payment successfully. There is a trade-off between security and routing performance (which is also discussed in [18]). If channel balance information becomes public information, substantial improvements in the success rate can be anticipated. However, in the current implementation focusing on security, the payment routing operation on the sender side is based on a trial-and-error approach and thus has a poor success rate. In the current implementation, if the selected shortest path fails because of insufficient funds on a specific channel, the sender removes the channel in the local view and finds another shortest path. The payment routing operation uses the above procedure repeatedly until finding a successful path.

2.6. Out-of-band channel

The initial invoice is not communicated over the PCN but over an out-of-band (OOB) channel. In Lightning Network, the payment invoices are presented as QR-code and sent by message channels [21]. Such an OOB channel can be further utilized to exchange information. In [22], the Spear method utilizes the OOB channel to communicate the corresponding pre-images of partial payment to avoid overdrawing. With the availability of the OOB channel, our Auto-Tune algorithm with OOB shares the last-hop balance information through the OOB channel. It improves our routing decision (detailed in Section 5.7). Such an approach can be treated as compromising the last hop balance of information security for improving the routing efficiency. We think this approach is a potential and viable solution because it only involves the two parties (sender and receiver) and improves the routing by better using the information. Such an approach does not entirely conflict with the initial assumption that channel balances should remain private because the channel balances along the routing paths are still unknown to the sender except the first and last hop balances. The receiver can decide whether it wants to share the last hop balance with the sender.

2.7. Hash-time-locked-contracts and onion routing

There are two main techniques in PCN for securing payment routing: HTLCs (Hash-Time-Locked-Contracts) and onion routing. First. HTLC guarantees payments to be securely routed across multiple payment channels. Specifically, HTLC ensures that the receivers acknowledge receiving the payment before a pre-agreed deadline (timelocks) by generating cryptographic proof of payment (hashlocks). The PCN then utilizes onion routing to securely and privately route HTLCs within the network. By using onion routing, each node on the path only knows its immediate predecessor and successor without knowing the actual payer and payee of this payment.

3. Motivation

As we mentioned in Section 2.5, the current implementation of PCN is based on single-path routing using a trial and error approach. Such single path routing using only the channel capacity information is expected to have a poor payment success probability. Therefore, the R&D community of PCN works on techniques for supporting multi-path payments such as Multi-path Payment (MPP) [11] or Atomic Multi-path Payments (AMP) [12] to improve the success rate. However, little work has investigated and proposed multi-path routing algorithms for PCN [13,23–26]. The Spider algorithm proposed in [23,24] requires the routers in PCN to control transaction rates, which is not how PCN is currently implemented. The state-of-the-art Flash [13] algorithm requires a threshold for separating the transactions into elephant and mice transactions and processing them differently. Setting up such a threshold beforehand is problematic in PCN because inferring the size of payments is challenging in an onion-routed network (also mentioned in [24]). In addition, they designed the Flash under the assumption that channel balance can be probed or known,[3] which violates the current design. It motivates us to present our Auto-Tune algorithm. Auto-Tune makes its own decision about whether a payment should be split or not (without setting a payment-based threshold) based on the dynamic and implicit network view with limited channel capacity information (which aligns with the current implementations of PCN that keep the channel balance as a secret for privacy concern). Moreover, our Auto-Tune algorithm uses all possible information to select the route better and improve the success rate. Below, we use a motivation example to show how multi-path payments can help the success rate.

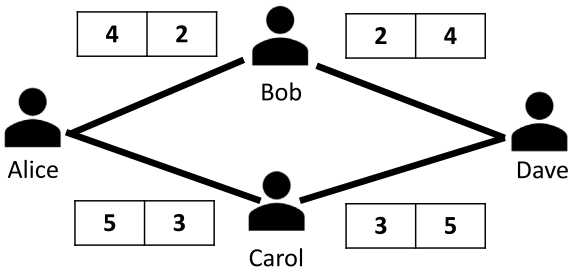


Fig. 1. Motivation example for multi-path approach.

Motivation Example: In Fig. 1, there are four entities in a small PCN. The values in the rectangle represent the corresponding channel balances in a snapshot. For instance, on the Alice–Bob Channel (C_{AB}), the channel balance on Alice's side $b_{AB} = 4$ and the channel balance on Bob's side $b_{BA} = 2$. If Alice wants to send four tokens to Dave, Alice will sequentially try the route Alice–Bob–Dave and the route Alice–Carol–Dave because the channel capacities on the corresponding channels are all higher than 4. Since the insufficient funds (b_{BD} and b_{CD}) on the channels C_{BD} and C_{CD}, both of these two routes get failed. However, suppose the multi-path routing is enabled. In that case, Alice can send two tokens through Carol, and two tokens through Bob, to make the payment succeed. By splitting a payment into fewer split amounts, a split payment is relatively easy to succeed on each selected path. More paths used for sending a payment could generate more routing fees. Because of this, the routing algorithm design should carefully consider the trade-off between the success rate and the routing fee. OOB channel can be available as described in Section 2.6. OOB channel, if available, can help with routing optimization because the OOB channel can share the balances of the last-hop channels, and the source can utilize the information for improved routing decisions. For example, Alice can use the last-hop balance information, $b_{BD} = 2$ and $b_{CD} = 3$ in Fig. 1, to help with the routing decision. For example, if Alice wants to send three tokens to Dave, Alice will select the route Alice–Carol–Dave rather than Alice–Bob–Dave to save the yielded routing fee. In our work, we analyze, compare, and show that the OOB channel is helpful for routing optimization in Section 6.

4. Model and assumptions

Current PCN implementations periodically share the network topology $G = (V, E)$ to all nodes by using gossiping protocols [7,27]. The shared contents of a node are its IP address, the routing fee model, and the channel capacity (rather than channel balance). Such information is stored on each node to help with the routing decision. As discussed in Section 1, the channel balances are private information for security reasons in current PCN implementations. Hence, our algorithm design follows such a current design which is an important requirement for PCN routing. We also assume that the channel balance

[3] They called channel balance as channel capacity in their paper.

Table 1
Notations.

Variables	Meaning
C_{ij}	The payment channel established by i and j
b_{ij}	The channel balance of C_{ij} on i's side
c_{ij}	The channel capacity of C_{ij}, $c_{ij} = b_{ij} + b_{ji}$
a_{ij}	A payment with amount a_{ij} sent from i to j
f_b, f_a	The base and additional fee for routing a payment on a node
$G = (V, E)$	A PCN Topology represented by G
$P = \{p_1, p_2, \dots, p_k\}$	A set with k candidate paths for forwarding a payment
$(c_x^1, c_x^2, \dots, c_x^n)$	The capacities sequence of p_x with n hops
$(b_x^1, b_x^2, \dots, b_x^n)$	The balances sequence of p_x with n hops
a_{ij}^x	A split payment amount of a_{ij}
$s(p_x, a_{ij}^x)$	The path score of p_x in forwarding a_{ij}^x
$s'(p_x, a_{ij}^x)$	The weighted path score of p_x in forwarding a_{ij}^x
$m(p_x)$	The maximum flow that can support by p_x
$\widetilde{P_f}$	The best feasible path set selected for tuning on split payment amounts
t_p	A probability-based threshold on single link
T_p	The finalized threshold for a path based on t_p
L, U	The lower and upper bound of channel balance inferred from the success/failure of a payment

distribution follows a uniform distribution (which has been shown as a reasonable assumption in [28]). However, our routing algorithm is generally applicable to other kinds of channel balance distribution. More specifically, we compute the success probability among candidate paths with the above assumptions. Since our routing algorithm is based on the success probability, the algorithm can still work by adjusting the operation in computing the probability if the balance distribution follows other distributions. We also assume that the implementation of multi-path payments such as AMP is achieved. In fact, several PCN implementations have already released a version facilitated with multi-path payments [12]. In the following section, we demonstrate our proposed Auto-Tune routing algorithm. We also summarize the main notations in Table 1.

5. Auto-tune routing algorithm

5.1. Candidate paths selection

For sending a payment, the sender i first computes k shortest paths as the candidate paths denoted as $P = \{p_1, p_2, \dots, p_k\}$. We use Yen's algorithm [29] to obtain k shortest paths in our experiment, which is also used in several works [13,20] and eclair[4] (one of the current Lightning Network implementations). The value of k can be adjusted according to the obtained success probability results using a different setting. Undoubtedly, more candidate paths recorded for sending a payment can increase the success rate by considering more paths. On the other hand, it also increases the opportunities to forward a payment through a longer path, which yields more routing fees. We carefully analyze this in our experiments in Section 6.

5.2. Score based on the success probability of a path

For all paths in P for sending a payment with amount a_{ij}, we first compute their path scores based on the success probability of the path. We denote $s(p_x, a_{ij}^x)$ as a function to output the score with the input candidate path $p_x \in P$ with a split payment amount $a_{ij}^x, a_{ij}^x \le a_{ij}$. We will discuss how we give initial a_{ij}^x for computing the score before a more comprehensive process for payment splitting in Section 5.4. Assume that the channel capacities sequence involved in p_x with n hops is $(c_x^1, \dots, c_x^n)$. The corresponding balances (unknown to the sender) sequence for sending the payment using p_x is $(b_x^1, \dots, b_x^n)$. The path score is computed as $s(p_x, a_{ij}^x) = \prod_{s=1}^{s=n} P(b_x^s \ge a_{ij}^x)$. In this work, we assume that the channel balances distribution on each link follows the uniform distribution hence we compute the $P(b_x^s \ge a_{ij}^x)$ by $\frac{c_x^s - a_{ij}^x + 1}{c_x^s}$.

5.3. Weighted path score using first-hop channel balance

In fact, we should better utilize the information on the first hop because sender i has not only the capacity information c_x^1 but also the exact channel balance b_x^1. Because we want to prioritize the paths based on the path score, such information can definitely help with a more accurate score among different paths. For instance, there are two paths, both with three hops and with the same channel capacities. The path with a higher first-hop balance should have a higher rank because it provides more flexibility for payment splitting and can possibly reach a higher success probability. To incorporate this information into the score, we propose a weighted path score, $s'(p_x, a_{ij}^x) = \prod_{s=1}^{s=n} min(m(p_x), a_{ij}) \cdot P(b_x^s \ge a_{ij}^x)$. $m(p_x) = min(b_x^1, c_x^1, c_x^2, \dots, c_x^n)$ representing the maximum flow that can support by p_x. We then take the smaller value between $m(p_x)$ and a_{ij} as the additional weight of the path score. This is because if the maximum flow of p_x is higher than a_{ij}, we only need to consider the case where it supports the entire a_{ij}.

5.4. Payment splitting

5.4.1. Autonomous payment splitting

As mentioned in Section 2.5, we want to consider both the success rate and the routing fee in our algorithm. Therefore, our algorithm finds the smallest number of paths while ensuring feasibility. Specifically, a feasible path set is defined as P_f satisfying the following condition, $\sum_{\forall p_x \in P_f} m(p_x) \ge a_{ij}$. Among all possible P_f, the algorithm retrieves those P_f with the smallest $|P_f|$ denoted as P_m. For each $P_m^n \in P_m$, a weighted path score can be computed as $s(P_m^n) = \prod_{\forall p_x \in P_m^n} s(p_x, a_{ij}^x)$. To compute the scores without knowing the best allocation a_{ij}^x on the path p_x, we use equal splitting to compute the weighted path scores, i.e., $a_{ij}^x = \frac{a_{ij}}{y}$ where y is the smallest $|P_f|$. We select the one with the highest path scores denoted as $\widetilde{P_f}$. By doing this, the algorithm will not blindly split the payments into too many paths rather than trying fewer paths to save the routing fee. If a payment with a small amount can be successfully processed by using only a single path ($|\widetilde{P_f}| = 1$), our Auto-Tune algorithm finds the path with the highest score (i.e., the highest success probability). Otherwise, the algorithm splits the payment ($|\widetilde{P_f}| > 1$).

5.4.2. Tuning the split payment amounts

Our Auto-Tune algorithm then focuses on $\widetilde{P_f}$ and tunes the best payment splitting on $\widetilde{P_f}$ with the highest success probability. To be more specific, finding $a_{ij}^x, \forall x, 1 \le x \le |\widetilde{P_f}|$, such that $\sum_{x=1}^{x=|\widetilde{P_f}|} a_{ij}^x = a_{ij}$ and $\prod_{\forall p_x \in \widetilde{P_f}} s(p_x, a_{ij}^x)$ is maximized. We also set up a probability threshold called T_p. If the highest success probability provided by the best payment splitting on $\widetilde{P_f}$ is less than T_p, the algorithm will select another feasible set with the next highest sum of scores until the probability is higher than T_p. Because the success probability will be affected by the total number of hops in $\widetilde{P_f}$ denoted as $h(\widetilde{P_f})$, we also consider it when we set up the threshold T_p computed as $T_p = (t_p)^{h(\widetilde{P_f})}$ where t_p is the acceptable probability on a single link. In the simulation, we will vary this t_p to see how it affects the success rate and the routing fee. We take this heuristic approach focusing on the payment splitting on $\widetilde{P_f}$ because the exhaustive search by checking all partitions on all candidate paths requires significant computational overhead. In

[4] https://github.com/ACINQ/eclair

some cases, the success probabilities of some candidate paths might be minimal, and it is not worth computing different payment partitions on those paths.

5.5. Score update after sending a payment

After sending the corresponding split payments on $\widetilde{P_f}$, the sender will receive notifications about whether each split payment is successfully processed or not. If a split payment is failed, the sender will know which channel on the path has an insufficient balance for forwarding the payment. Our algorithm utilizes this information to compute a more accurate $P(b_x^s \geq a_{ij}^e)$ (where a_{ij}^e is the equal split payment used for score computation) because the upper bound of b_x^s (such upper bound information will be recorded as U in Algorithm 1) is updated from c_x^s to $a_{ij}^x - 1$ (where a_{ij}^x is the amount of the failed split payment). In contrast, if a split payment a_{ij}^x succeeds, the sender knows that some channel balances have been utilized. Therefore, a more accurate $P(b_x^s \geq a_{ij}^e)$ can also be computed because the upper bound is updated from c_x^s to $c_x^s - a_{ij}^x$. Moreover, suppose a path failed, but some links on the path have efficient amounts to support a_{ij}^x. In that case, Our algorithm will all record it as lower bound information. With this information, $P(b_x^s \geq a_{ij}^e)$ could be one if $a_{ij}^x \geq a_{ij}^e$ because $b_x^s \geq a_{ij}^x$. The payment demand a_{ij} will be updated accordingly if some split payments succeed. The routing algorithm then goes back to the payment splitting for finding the next $\widetilde{P_f}$ mentioned in Section 5.4 until $a_{ij} = 0$.

5.6. Using the OOB channel for sharing last-hop balances

As we mentioned in Section 2.6, for the real-world implementation of PCNs, there is an out-of-band (OOB) channel between two parties for exchanging an invoice. We can use such an OOB channel to obtain the last-hop channel balance information from the receiver. By observing the last-hop balances, the algorithm's performance and the success probability can be anticipated to be further optimized. The weighted path score is more accurate and useful for payment splitting. To be more specific, $s'(p_x, a_{ij}^x) = \prod_{s=1}^{s=n} min(m(p_x), a_{ij}) \cdot P(b_x^s \geq a_{ij}^x)$ where $m(p_x) = min(b_x^1, c_x^1, c_x^2, \ldots, b_x^n)$. Since, c_x^n is substituted by b_x^n, a more accurate (bounded) weight $min(m(p_x), a_{ij})$ can be obtained. In addition, we can filter out more useless candidate paths if $b_x^n < a_{ij}^x$ and makes $P(b_x^n \geq a_{ij}^x) = 0$. We also conduct the performance comparison between Auto-Tune without the OOB channel and Auto-Tune with the OOB channel in our simulation.

5.7. Flow and analyses of the auto-tune routing algorithm

To help better understand the operational flow of the Auto-Tune routing algorithm, we summarize the operations in pseudo-codes in Algorithms 1. When a sender node uses the algorithm to make a routing decision for payment with amount a_{ij}, the node uses the latest obtained network topology G and the lower L and upper bound U information on each channel as the inputs of the algorithm. A channel's lower and upper bound is initially set to zero and the channel capacity, respectively. The sender nodes should also specify the desired threshold (t_p) and the number of candidate paths (k) as two other algorithm inputs. We also analyze how the setting of these control parameters can affect the success probability in Section 6.2. The algorithm's output is a binary factor, s, showing the result of success or failure.

The algorithm first uses Yen's algorithm [29] to compute P with the time complexity $O(kn(m + nlogn))$ where n and m are the numbers of vertices and edges in G respectively (Line 1). If the payment has not been fulfilled yet, i.e., $a_{ij} \neq 0$, the algorithm starts the process for the path selection and the payment splitting. The algorithm computes the corresponding P_m, i.e., the feasible path sets with the smallest $|P_f|$, and we use a maximum priority queue for P_m using the score of $s(P_m^x)$ as its priority. While P_m is not empty, the one with the highest score is popped out as $\widetilde{P_f}$. Considering that the priority queue is implemented using a sorted array. The time complexity for sorting is $O(nlogn)$, where n is the size of P_m. After getting $\widetilde{P_f}$, the algorithm finds the best allocation of a_{ij}^x reaching the highest score using the brute-force search. However, the split payments will be sent out only if the computed score (based on probability) is higher than a pre-defined threshold. If the threshold is not satisfied and P_m is not empty, the algorithm will obtain an updated $\widetilde{P_f}$ for further processing. After sending out a_{ij}^x (Line 9), the sender waits for the success and failure results from the involving intermediate nodes. We use a flag f to indicate whether split payments are determined and sent out, so we set $f = 1$ (Line 10). The performance is thus impacted by the delay (including network delay and processing delay). The algorithm uses the result to update the corresponding U, L (in order to compute more accurate scores for the next round), and a_{ij} (Line 11). After getting all the results of the split payments, if the payment demand $a_{ij} \neq 0$, the algorithm does the same operations for path selection and payment splitting based on the updated a_{ij} until $a_{ij} = 0$. However, if all the path sets in P_m are tested, and none of them reach the pre-defined threshold making $f = 0$, we stop the process (break out of the while loop) immediately (Line 13–Line 14). The finalized output s is then determined (Line 15–Line 18). If the a_{ij} becomes zero, it means that payment succeeded ($s = 1$). Otherwise, the payment failed ($s = 0$).

Algorithm 1: Auto-Tune Routing Algorithm.

Input: A payment with amount a_{ij}, t_p, G, U, L, k
Output: s (a binary factor to show success/failure)

```
1: Compute P = {p_1, p_2, ..., p_k};
2: while a_ij ≠ 0 do
3:     Obtain P_m
4:     f = 0;
5:     while P_m is not empty do
6:         P̃_f = P_m.pop()
7:         Find a_ij^x maximizing ∏_{∀p_x∈P̃_f} s(p_x, a_ij^x)
8:         if ∏_{∀p_x∈P̃_f} s(p_x, a_ij^x) ≥ (t_p)^{h(P̃_f)} then
9:             Send a_ij^x
10:            f = 1
11:            Use the success/failure results to update U, L, a_ij
12:            break
13:    if f = 0 then
14:        break
15: if a_ij = 0 then
16:    s = 1
17: else
18:    s = 0
```

6. Simulation

We evaluate the performance of our Auto-Tune algorithms against other algorithms based on simulation. The simulations are implemented in Matlab. For simplicity, we omit some aspects of the real-world payment procedure, such as the HTLC protocol. In Section 6.1, we describe the simulation setup. In Section 6.2, we perform analyses for the selection of the control parameters (those the algorithm can control), while Section 6.3 is about the analyses of environmental parameters (those provided by the system implementation, e.g., channel capacity, or by the environment outside of the relay node, e.g., the clients/payment). Sections 6.2 and 6.3 are based on one synthetic topology. Section 6.4 then focuses on the performance results among all topologies while using a more realistic setting for channel capacities.

6.1. Simulation setup

6.1.1. Network topology

We first generate two synthetic network topologies for our simulation. One follows the Watts–Strogatz (WS) small-world graph [30]

with $|V| = 50$ and $|E| = 200$. The other one is a scale-free Barabási–Albert (BA) graph [31] with $|V| = 50$, and the average degree of nodes is set to 5. In the scale-free BA graph, low degree nodes tend to connect to high degree nodes rather than low degree ones, which aligns with the attachment strategy in the current implementation [32]. Specifically, the BA graph is based on the preferential attachment rule, i.e., nodes are generated and attached to other nodes with higher node degrees in higher probability. We select these two synthetic network topologies because the authors in [33] confirm that Lightning Network can be classified as a small-world and scale-free network. To further investigate the performance in real-work Lightning network topology, we use a snapshot of Lightning Network taken on 21st September 2020, consisting of 7498 nodes and 37097 edges collected from a GitHub repository [34].

6.1.2. Channel balance setting

For setting the channel balances and capacities, we generate random capacities c_{ij} for each link. The balance of b_{ij} is then randomly selected between $[0, c_{ij}]$. We compute $b_{ji} = c_{ij} - b_{ij}$ accordingly. For the analyses conducted in Section 6.3, the channel capacities are uniformly selected from given ranges to observe the effect when channel capacities are increased. To make the setting closer to the real world, we obtain the cumulative distribution function (CDF) of capacities measured by [6]. c_{ij} is then randomly generated and assigned based on the CDF in Sections 6.2 and 6.4.

6.1.3. Payment setting

We set the default payment amount as the macro payment of 10^6 satoshis defined in [35] (also used in [32]). We use this payment to better evaluate the performance of our scheme that can autonomously split into multiple paths if a single path cannot support such macro payments. The sender and the receiver nodes are randomly chosen from V for sending each payment.

6.1.4. Routing algorithms

We compare our algorithms (including Auto-Tune with OOB and Auto-Tune without OOB) with the algorithm based on the shortest path approach and the state-of-the-art Flash [13] approach (a probing approach knowing the exact channel balance which conflicts with the current implementation). We set the size of the candidate path set equally for different algorithms based on our k selection conducted in Section 6.2, to make a fair comparison. For the routing fee analyses in Section 6.4.2, we assume all nodes use the default fee model mentioned in Section 2.5.

6.1.5. Performance metrics

We select two metrics for our simulation to compare the performances among different routing algorithms. The first metric is the success probability, i.e., the number of completed payments over the number of generated payments. The second metric is the fee ratio comparison of those successful payments using multiple paths. All the results are measured among 10^3 payments.

6.2. Analyzing the control parameters

We first evaluate the success probability using the Auto-Tune with OOB algorithm using WS small-world graph while varying k and t_p in Fig. 2. A bigger k allows more candidate paths to be considered in our algorithm, thus increasing the success rate accordingly. Similarly, a lower t_p represents a smaller threshold T_p used for filtering out the feasible path sets, i.e., more paths will be left for consideration. So, the overall success probability is higher by trying more possible path sets. Fig. 2 shows the trend we mentioned above. We select k and t_p based on the curve we obtained. Specifically, if increasing the value of k or t_p cannot significantly improve success probability (i.e., the curve is relatively flat), we will not select bigger values for them. Based on the results in Fig. 2, we select $k = 80$ and $t_p = 1/4$ accordingly. Similar analyses are conducted for setting k and t_p for our Auto-Tune based algorithms under different network topologies. In this work, we only include the analyses on Auto-Tune with OOB using WS small-world graph. Auto-Tune with OOB and without OOB might select different k. Since we want to select the same k for all routing algorithms for making a fair comparison, We use the k we select for Auto-Tune with OOB as the finalized k.

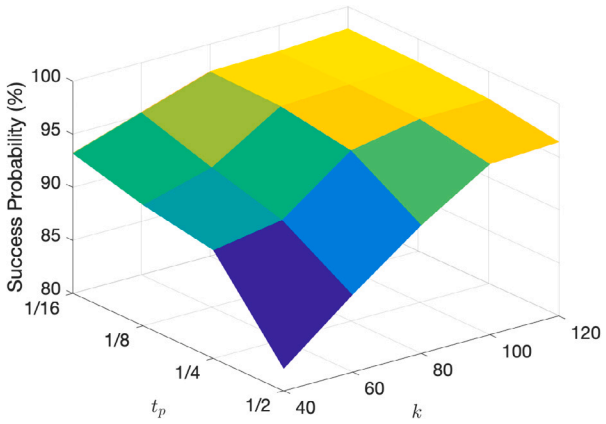


Fig. 2. Analyzing the control parameters.

6.3. Analyzing the environmental parameters

6.3.1. Varying channel capacities

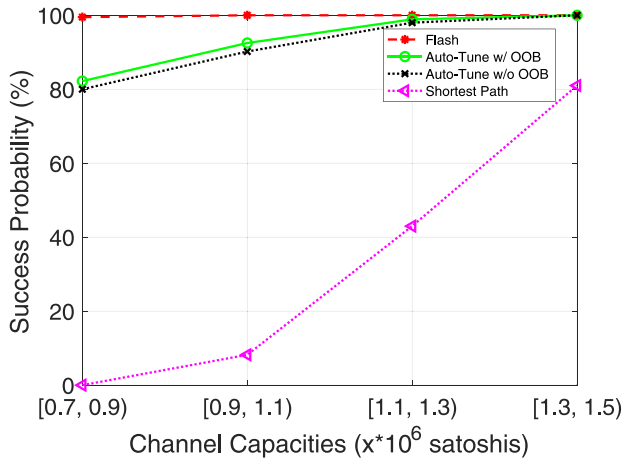
We use the default payment amount (10^6 satoshis) and vary the channel capacities. Specifically, the channel capacities of each channel are randomly chosen from different ranges. As shown in Fig. 3(a), when we increase the channel capacities, more paths in the network can support the payment, and the success rate increases. However, by using the shortest path approach, the success rate is only 81% if the capacities are selected from $[1.3 \times 10^6, 1.5 \times 10^6)$ satoshis. In contrast, both of our Auto-Tune based algorithms can reach 100%. By the information shown in [6], the median value of channel capacity is 10^6 satoshis. So, we focus on the results where the channel capacities are selected from $[0.9 \times 10^6, 1.1 \times 10^6)$. Under this case, our Auto-Tune with OOB can still reach 92.5% success rate and significantly outperforms the shortest path approach with only 8.2% success rate.

6.3.2. Varying payment amounts

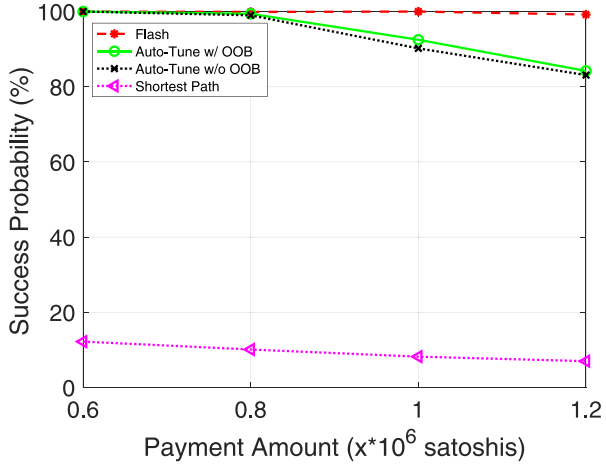
When varying the payment amount with a fixed range of channel capacities $[0.9 \times 10^6, 1.1 \times 10^6)$, we can anticipate that the more payment amount, the less the success rate in this case. The result in Fig. 3(b) successfully validates this. Our Auto-Tune without OOB algorithm can still have an 84.2% success rate if the payment is set to 1.2×10^6 satoshis. In contrast, the shortest path approach only has 7% success rate. It shows that a careful selection of candidate paths and payment splitting accomplished by Auto-Tune based algorithms significantly improve the success rate if payment is large and cannot be fulfilled by using a single path approach. Even though the Flash algorithm reaches a higher success probability, it violates the current implementation by utilizing the exact balance information.

6.4. Performance comparison

For better setting the channel capacities, we randomly generate channel capacities according to the cumulative distribution function (CDF) of capacities (obtained from public Lightning Network statistics [6]) by inverse transform sampling for the following simulation, including all network topologies.



(a) Varying Channel Capacities



(b) Varying Payment Amounts

Fig. 3. Analyzing the environmental parameters.

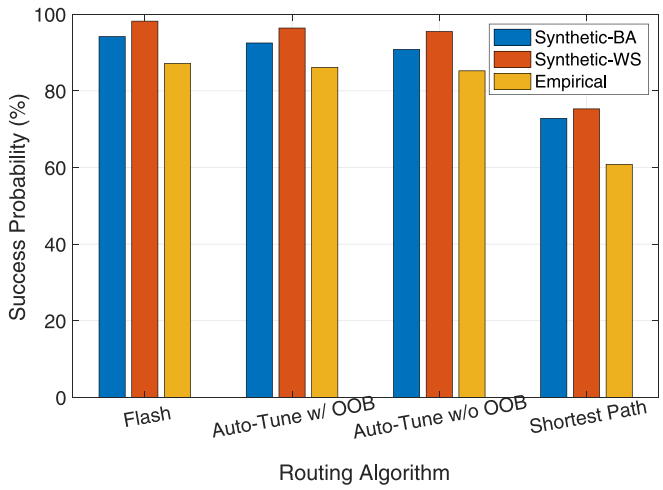


Fig. 4. Success probability.

Table 2
Fee ratio.

	WS	BA	Empirical
Auto-Tune w/ OOB vs. Flash	1.1472	1.1141	1.1232
Auto-Tune w/o OOB vs. Flash	1.3178	1.1823	1.2314

6.4.1. Success probability

Fig. 4 shows the result of the success probabilities among different routing algorithms using different topologies. One can observe two Auto-Tune algorithms outperforms the shortest path approach in all topologies. The Auto-Tune with OOB with more information to compute more accurate path scores performs better than the Auto-Tune without OOB. The success probabilities using the empirical data (real-world topology) are worse than that of synthetic topologies because of the significant difference in network scales. The shortest path approach only has a 60.83% success probability because it is more challenging to satisfy a payment using a single path approach in this network scale. In contrast, our Auto-Tune with OOB algorithm can still get 86.15% success probability using the multi-path approach while using both first-hop and last-hop information to select the paths better. We compare Flash and Auto-Tune with OOB to highlight that our performance is close to Flash (using the exact balance information and violating the requirement of current PCN implementations). Specifically, among those successful payments achieved by Flash, 98.17%, 98.13%, and 98.83% of the payments can also succeed using Auto-Tune with OOB (using the channel capacities, which is compatible with the current implementations) for BA graph, WS graph, and real-world topology, respectively.

6.4.2. Fee ratio

Since Flash finds the optimal (minimum) fee, we take it as the denominator to compute the fee ratio. We focus on the comparison between Auto-Tune based algorithms and Flash. This is because those payments that can succeed using the Shortest Path algorithm can also be satisfied by Flash and Auto-Tune based algorithms by using single path routing. Since this work focuses on the multi-path approach, we mainly compare Auto-Tune based algorithms and Flash in the case that more than one path is selected to accomplish a payment. The results we obtained for the average fee ratio of Auto-Tune with OOB and Auto-Tune without OOB are shown in Table 2. Again, the last hop information used in Auto-Tune with OOB has better results in all topologies. This is because a better selection of paths can decrease the number of paths required for the payment and thus decrease the routing fee.

7. Related work

7.1. PCN routing

The routing algorithms for PCNs remain an open problem and an active research area. The white paper of Lightning Network [36] only suggests routing a path using a BGP-like system. Current implementations of PCN keep the balance a secret for privacy concerns. In [5], authors discussed that disclosing the balance information may allow an attacker to conduct attacks such as Lockdown attack [10] and payment retrospect [20]. Unfortunately, most of the works for payment routing did not consider this in their designs. Lin et al. [37] proposed a fund's skewness-aware transaction routing algorithm to route transactions based on a single path approach. They attempt to increase the success probability while reducing the fund's skewness on channels. The skewness metric is computed based on the balance information. Sivaraman et al. proposed a Spider Network [23,24], a new PCN assuming each pair of PCN nodes has a particular transaction rate. However, Spider requires the routers to control the rates, which is not how PCN is currently implemented. In [13], authors assume the balance can be probed, separate the transactions into elephant and mice transactions and process them differently. However, how to decide an elephant-mice threshold for a sender is problematic because inferring the payment amounts and using it for setting up the elephant-mice threshold is hard in PCN (also mentioned in [24]). We

proposed our Auto-Tune algorithms to autonomously decide whether a payment should be split without using a pre-defined threshold. Rene et al. [28] also proposed using a probabilistic model to make the multi-path routing decision. However, they only consider equal payment splitting instead of tuning the splitting amount to optimize the success probability that Auto-Tune conducts. Moreover, Auto-Tune utilizes the last-hop balance information exchanged through the OOB channel to optimize the success probability further.

Atomic Multi-path Payments (AMP) [12] is a newly implemented technique to ensure that the receiver will not be paid until all partial payment flows are completed. Since the transfer delays, if any partial payment is failed or delayed, some works propose using redundant paths to improve the success probability further. Bangria et al. [38] propose Boomerang using secret-sharing to recover the overdrawing situation. Sonbol and Majid [22] propose Spear, an improved method that outperforms Boomerang with lower delay and lower computational overhead. However, such an approach requires the sender to send the payments with a payment amount larger than the original amount. In other words, those additional amounts will be locked in the PCN before the overdrawing issue is resolved. Therefore, the approach is questionable in practice because it contradicts the usual payment behavior.

7.2. Revealing the balances

Some works focus on the security issue of revealing the balance. In [39], authors proposed that attackers can execute multiple fake payments to measure an unknown balance of a payment channel. Authors in [5] presented a novel non-intrusive balance tomography method to infer the balances by performing legal transactions between two pre-created Lightning Network nodes. Although several works focus on revealing the balance, a standard sender will not take the effort to infer the balance before sending out a payment. However, a sender can use the success/failure result of sending a payment to help the following routing decision and improve the success probability of the payment, which is what we have done in this work.

7.3. Related flow problems

7.3.1. Minimum cost flow problem:

The minimum cost flow (MCF) problem [40] finds the cheapest way of sending a certain amount of flow from source to destination on a given network. Each link in the network has its respective capacity and cost. For making an analogy between MCF and PCN routing, the source is the sender node, and the destination is the receiver node. The capacity of a link is the respective outgoing balance. In addition, the cost model is also different between MCF and PCN routing. For MCF, the cost model is proportional, i.e., the cost of the flow on each link is proportional to the amount of that flow. In contrast, the cost model in PCN routing is a linear cost function mentioned in Section 2.5 with a base fee and an additional fee. The base fee will not proportionally increase when the flow amount increases. Therefore, we cannot solve a PCN routing by modeling it as an MCF problem.

7.3.2. Multi-commodity flow problem

A payment routing is considered from the sender's perspective. Suppose we consider the payment routing from a network perspective (i.e., a centralized third party gathering all the payment information on a PCN). The routing problem can be treated as an All-or-nothing multi-commodity flow problem [41]. In a Multi-commodity flow problem, multiple commodities (i.e., flow demands) are sent from different source nodes and sink nodes in a network. The All-or-nothing requirement ensures that all the commodities are sent to the receiver node atomically. The commodities here can be treated as payments in PCN. However, similar to the judgment above, the links' capacity information is known only if the channel balance information stays public, which violates the current design. A centralized third party also conflicts with the decentralized nature of cryptocurrency. In other words, solving it as a multi-commodity flow problem is only acceptable under a private payment network where the balance information can be shared and solved in a centralized manner. In [42], Enes et al. proposed a private payment network formed by retailers and modeled it as a network optimization problem inspired by multi-commodity network flow problems. Auto-Tune focuses on routing for a public payment network; hence it cannot be solved in a centralized manner.

8. Conclusion

The design of routing algorithms for PCNs based on multi-path routing is an active research area. The state-of-art Flash algorithm [13] assumes that the channel balance can be probed and thus violates the current PCN implementations. We proposed our Auto-Tune algorithm satisfying the significant privacy requirement for PCN routing that channel balance should be kept secret, which is consistent and compatible with current PCN implementations. Auto-Tune autonomously splits a payment based on the dynamic and implicit network view with channel capacities and optimally tunes the payment splitting amounts to maximize the success probability. Because Auto-Tune makes the routing decision based on the success probability of candidate paths, it is generally applicable to other channel balance distributions while we assume a uniform distribution in this work. For further improvement, we proposed using the OOB channel to share the last-hop balance information and optimize the routing decision. The simulation result shows that our Auto-Tune with OOB outperforms the shortest path approach with 1.41 times better performance on success probability using real-world Lightning Network topology. Among the successful payments achieved by Flash, 98.83% of the payments can also succeed using Auto-Tune with OOB.

CRediT authorship contribution statement

Hsiang-Jen Hong: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing, Visualization. **Sang-Yoon Chang:** Writing – review & editing, Supervision, Funding acquisition. **Xiaobo Zhou:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

This research was supported by Colorado State Bill 18-086 and by the National Science Foundation, United States under Grant No. 1922410. The previous version of this work has been published in IEEE Conference on Local Computer Networks (LCN) in 2022 [43]. Our extension and enhancement from that previous version include: (1) Providing a more comprehensive PCN background. (2) Adding a motivation section facilitated with a motivation example to show why multi-path payment is essential to the PCN routing and how the OOB channel can benefit the routing. (3) Utilizing the OOB channel to enhance our Auto-Tune algorithm by sharing the last hop balance information, which helps to improve the routing decision and performance. (4) Adding control parameter analyses (to establish the parameter setting of Auto-Tune better) and environmental parameter analyses (to examine how it affects the success probability of Auto-Tune. (5) Providing a more solid related work section by introducing the current attacks revealing the balances and related flow problems if the routing can be solved in a centralized approach.

References

[1] Bitcoin Core, https://bitcoin.org/en/bitcoin-core/.
[2] Ethereum, https://ethereum.org/en/.
[3] The lightning network, https://lightning.network/.
[4] The raiden network, https://raiden.network/.
[5] Y. Qiao, K. Wu, M. Khabbazian, Non-intrusive and high-efficient balance tomography in the lightning network, in: Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security, ASIA CCS '21, 2021, pp. 832–843.
[6] Lightning network search and analysis engine, https://1ml.com/statistics.
[7] Lightning network daemon, https://github.com/lightningnetwork/lnd.
[8] c-lightning, https://github.com/ElementsProject/lightning.
[9] eclair, https://github.com/ACINQ/eclair.
[10] C. Perez-Sola, A. Ranchal-Pedrosa, J. Herrera-Joancomarti, G. Navarro-Arribas, J. Garcia-Alfaro, Lockdown: Balance availability attack against lightning network channels, in: Proceedings of the International Conference on Financial Cryptography and Data Security, 2020, pp. 245–263.
[11] Multiple-Path Payment (MPP), routing: multi-part MPP send, https://github.com/lightningnetwork/lnd/pull/3967/.
[12] AMP: Atomic multi-path payments over lightning, https://docs.lightning.engineering/lightning-network-tools/lnd/amp.
[13] P. Wang, H. Xu, X. Jin, T. Wang, Flash: Efficient dynamic routing for offchain networks, in: Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies, CONEXT, 2019, pp. 370–381.
[14] J. Xie, F.R. Yu, T. Huang, R. Xie, J. Liu, Y. Liu, A survey on the scalability of blockchain systems, IEEE Netw. 33 (5) (2019) 166–173, http://dx.doi.org/10.1109/MNET.001.1800290.
[15] D. Yang, C. Long, H. Xu, S. Peng, A review on scalability of blockchain, in: Proceedings of the 2020 the 2nd International Conference on Blockchain Technology, ICBCT '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 1–6.
[16] Q. Zhou, H. Huang, Z. Zheng, J. Bian, Solutions to scalability of blockchain: A survey, IEEE Access 8 (2020) 16440–16455, http://dx.doi.org/10.1109/ACCESS.2020.2967218.
[17] S. Kim, Y. Kwon, S. Cho, A survey of scalability solutions on blockchain, in: Proceedings of the 2018 International Conference on Information and Communication Technology Convergence, ICTC, 2018, pp. 1204–1207, http://dx.doi.org/10.1109/ICTC.2018.8539529.
[18] W. Tang, W. Wang, G. Fanti, S. Oh, Privacy-utility tradeoffs in routing cryptocurrency over payment channel networks, in: Proceedings of the 2020 SIGMETRICS/Performance Joint International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS '20, 2020, pp. 81–82.
[19] N. Papadis, L. Tassiulas, Blockchain-based payment channel networks: Challenges and recent advances, IEEE Access 8 (2020) 227596–227609, http://dx.doi.org/10.1109/ACCESS.2020.3046020.
[20] G. Kappos, H. Yousaf, A. Piotrowska, S. Kanjalkar, S. Delgado-Segura, A. Miller, S. Meiklejohn, An empirical analysis of privacy in the lightning network, in: Proceedings of the Financial Cryptography and Data Security, Springer International Publishing, 2021, pp. 167–186.
[21] A.M. Antonopoulos, O. Osuntokun, R. Pickhardt, Mastering the Lightning Network, O'Reilly, 2021.
[22] S. Rahimpour, M. Khabbazian, Spear: Fast multi-path payment with redundancy, in: Proceedings of the 3rd ACM Conference on Advances in Financial Technologies, AFT '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 183–191.
[23] V. Sivaraman, S.B. Venkatakrishnan, M. Alizadeh, G. Fanti, P. Viswanath, Routing cryptocurrency with the spider network, in: Proceedings of the 17th ACM Workshop on Hot Topics in Networks, HotNets '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 29–35.
[24] V. Sivaraman, S.B. Venkatakrishnan, K. Ruan, P. Negi, L. Yang, R. Mittal, G. Fanti, M. Alizadeh, High throughput cryptocurrency routing in payment channel networks, in: Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 20, 2020, pp. 777–796.
[25] G. Di Stasi, S. Avallone, R. Canonico, G. Ventre, Routing payments on the lightning network, in: Proceedings of the 2018 IEEE International Conference on Internet of Things (IThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data, SmartData, 2018, pp. 1161–1170.
[26] S. Roos, P. Moreno-Sanchez, A. Kate, I. Goldberg, Settling payments fast and private: Efficient decentralized routing for path-based transactions, in: Proceedings of the 2018 Network and Distributed System Security Symposium, NDSS, 2018, http://dx.doi.org/10.14722/ndss.2018.23254.
[27] Raiden network daemon, https://github.com/raiden-network/raiden.
[28] R. Pickhardt, S. Tikhomirov, A. Biryukov, M. Nowostawski, Security and privacy of lightning network payments with uncertain channel balances, 2021, arXiv, arXiv:2103.08576.
[29] J.Y. Yen, Finding the K shortest loopless paths in a network, Manage. Sci. 17 (11) (1971) 712–716.
[30] D.J. Watts, S.H. Strogatz, Collective dynamics of 'small-world' networks, Nature 393 (1998) 442.
[31] A.-L. Barabási, R. Albert, Emergence of scaling in random networks, Science 286 (5439) (1999) 509–512.
[32] K. Lange, E. Rohrer, F. Tschorsch, On the impact of attachment strategies for payment channel networks, in: Proceedings of the 2021 IEEE International Conference on Blockchain and Cryptocurrency, ICBC, 2021, pp. 1–9, http://dx.doi.org/10.1109/ICBC51069.2021.9461104.
[33] E. Rohrer, J. Malliaris, F. Tschorsch, Discharged payment channels: Quantifying the lightning network's resilience to topology-based attacks, in: Proceedings of the 2019 IEEE European Symposium on Security and Privacy Workshops, EuroS&PW, 2019, pp. 347–356.
[34] A. Mizrahi, Lightning-network-congestion-attacks, 2021, GitHub Repository, GitHub, https://github.com/ayeletmz/Lightning-Network-Congestion-Attacks.
[35] O. Ersoy, S. Roos, Z. Erkin, How to profit from payments channels, in: J. Bonneau, N. Heninger (Eds.), Proceedings of the Financial Cryptography and Data Security, Springer International Publishing, 2020, pp. 284–303.
[36] J. Poon, T. Dryja, The bitcoin lightning network: Scalable off-chain instant payments, 2016.
[37] S. Lin, J. Zhang, W. Wu, FSTR: Funds skewness aware transaction routing for payment channel networks, in: Proceedings of the 2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN, 2020, pp. 464–475, http://dx.doi.org/10.1109/DSN48063.2020.00060.
[38] V. Bagaria, J. Neu, D. Tse, Boomerang: Redundancy improves latency and throughput in payment-channel networks, in: Proceedings of the 24th International Conference on Financial Cryptography and Data Security FC, 2020, pp. 304–324.
[39] J. Herrera-Joancomartí, G. Navarro-Arribas, A. Ranchal-Pedrosa, C. Pérez-Solà, J. Garcia-Alfaro, On the difficulty of hiding the balance of lightning network channels, in: Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security, in: Asia CCS '19, New York, NY, USA, 2019, pp. 602–612.
[40] J. Orlin, A faster strongly polynomial minimum cost flow algorithm, in: Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing, STOC '88, 1988, pp. 377–387.
[41] C. Chekuri, S. Khanna, F.B. Shepherd, The all-or-nothing multicommodity flow problem, in: Proceedings of the Thirty-Sixth Annual ACM Symposium on Theory of Computing, STOC '04, 2004, pp. 156–165.
[42] E. Erdin, M. Cebe, K. Akkaya, S. Solak, E. Bulut, S. Uluagac, A bitcoin payment network with reduced transaction fees and confirmation times, Comput. Netw. 172 (2020) 107098, http://dx.doi.org/10.1016/j.comnet.2020.107098, URL https://www.sciencedirect.com/science/article/pii/S1389128619308850.
[43] H.-J. Hong, S.-Y. Chang, X. Zhou, Auto-tune: efficient autonomous routing for payment channel networks, in: 2022 IEEE 47th Conference on Local Computer Networks (LCN), 2022, pp. 347–350, http://dx.doi.org/10.1109/LCN53696.2022.9843633.

Hsiang-Jen Hong received the Ph.D. degree from the Department of Computer Science and Information Engineering at National Taiwan University of Science and Technology, Taiwan, in 2018. He is currently a post-doctoral research associate at the University of Colorado, Colorado Springs. His research interests are computer networking, blockchain, cybersecurity, and combinatorial optimization.

Sang-Yoon Chang (M'14) received the B.S. and Ph.D. degrees from the Department of Electrical and Computer Engineering at University of Illinois at Urbana-Champaign in 2007 and 2013, respectively. He is an associate professor at the Computer Science Department at University of Colorado Colorado Springs. He was a postdoctoral fellow with the Advanced Digital Sciences Center. His research is in security, networking, wireless, cyber–physical systems, and applied cryptography.

Xiaobo Zhou obtained the B.S., M.S., and Ph.D. degrees in Computer Science from Nanjing University, in 1994, 1997, and 2000, respectively. Currently he is a professor of the Department of Computer Science, University of Colorado, Colorado Springs. His research lies in Cloud computing and datacenters, BigData parallel and distributed processing, autonomic and sustainable computing. He was a recipient of the NSF CAREER Award in 2009.